

ATUALIZAÇÃO DE LIBS

Plano de Ação

Atualização de dependências do monorepo Lello

Diagnóstico, blockers, agrupamento por risco e execução em ondas

Versão: 1.0

Data: 12/07/2026

Escopo: 7 projetos do monorepo `app-lello-morar`

Ambiente atual: Flutter 3.38.9 · Dart 3.10.8

Pacotes analisados: 246 no grafo de dependências

Esforço estimado: 60 a 80 horas (7 ondas)

Status: aguardando aprovação para iniciar Onda 0

Sumário

1. Diagnóstico — dados reais coletados
2. Por que a arquitetura importa aqui
3. Três blockers a resolver antes de qualquer bump
4. Os 38 breaking changes agrupados por bloco
5. Plano de execução em 7 ondas
6. Critérios de aceite por onda
7. Riscos principais
8. Recomendação e próximos passos
9. Anexo — comandos úteis

1. Diagnóstico — dados reais coletados

Executado `flutter pub outdated` no `app-lello-essentials` (o pacote hub do qual todos os apps herdam via `path:`) e cruzado com contagem de uso real no código-fonte de todos os repositórios.

PACOTES NO GRAFO

246

SALTOS MAJOR

73

SALTOS MINOR

85

SALTOS PATCH

65

DEPS DIRETAS NO
ESSENTIALS

115

DIRETAS COM
BREAKING CHANGE

38

BLOCKERS
ESTRUTURAIS

3

ONDAS DE EXECUÇÃO

7

Leitura dos números

Dos 246 pacotes no grafo, apenas **115 são declarados diretamente** — o resto é transitivo e sobe junto. O trabalho real está nos **38 pacotes diretos com salto breaking**, que exigem análise de changelog e ajuste de código.

2. Por que a arquitetura importa aqui

```
app-lello-essentials (115 deps diretas – TODAS as libs moram aqui)
```

```
↑ path:
```

```
app-lello-shared
```

```
(só depende de essentials)
```

```
↑ path:
```

```
graph TD; Root[ ] --- sindico; Root --- morar; Root --- colaborador; Root --- morar-viver; Root --- sindico-viver;
```

sindico morar colaborador morar-viver sindico-viver

Consequência prática

- **90% do trabalho é em um único arquivo:** `essentials/pubspec.yaml` .
- **O impacto de um bump errado é em 5 apps ao mesmo tempo.** Não existe rollout gradual por app — se o `essentials` sobe, todos sobem.
- `morar-viver` e `sindico-viver` são white-labels raramente testados, mas herdam tudo. Precisam entrar no smoke test.

3. Três blockers a resolver antes de qualquer bump

Blocker 1 · Divergência de Java entre os projetos

BLOQUEANTE

PROJETO	JAVA	COMPILESDK
sindico	1.8 ⚠	36 (hardcoded)
morar	17	flutter.compileSdkVersion
colaborador	17	flutter.compileSdkVersion

O `sindico` está em **Java 1.8**. Firebase 4.x e a maioria dos plugins Android modernos exigem Java 11+ (na prática 17). **Sem subir o `sindico` para Java 17, o bloco Firebase não compila.**

Ação: alinhar `sourceCompatibility` / `jvmTarget` para 17 e usar `flutter.compileSdkVersion` em vez do 36 fixo.

Blocker 2 · Overrides obsoletos no `shared`

BLOQUEANTE

```
# app-lello-shared/pubspec.yaml
dependency_overrides:
  analyzer: 6.4.1      # essentials já resolve 7.7.1
  source_gen: 1.5.0    # essentials já resolve 2.0.0
  dart_style: ^2.3.7
```

Esses pins estão **travando o ecossistema de code generation** (`build_runner` , `drift_dev` , `chopper_generator` , `json_serializable`). Enquanto existirem, não dá para subir os geradores. Provavelmente são resquício de um conflito antigo já resolvido.

Ação: remover os pins, rodar build e verificar se o conflito original ainda se manifesta.

Blocker 3 · Dois pacotes travados sem caminho de upgrade

INVESTIGAR

`pub outdated` mostra `resolvable == current` para estes, ou seja alguma restrição os impede de subir:

PACOTE	ATUAL	RESOLVÍVEL	ÚLTIMA
<code>share_plus</code>	12.0.2	12.0.2 (travado)	13.3.0
<code>open_file</code>	3.5.11	3.5.11 (travado)	4.0.0

Ação: rodar `flutter pub deps` para identificar quem restringe a versão.

4. Os 38 breaking changes agrupados por bloco

Ordenado por risco × uso real. A coluna "Arquivos" conta quantos `.dart` importam o pacote em todo o monorepo.

Bloco A · Firebase — 11 pacotes, sobem JUNTOS

RISCO ALTO

PACOTE	ATUAL → ALVO	ARQUIVOS
<code>firebase_core</code>	3.15.2 → 4.13.0	24
<code>firebase_crashlytics</code>	4.3.10 → 5.2.7	30
<code>firebase_messaging</code>	15.2.10 → 16.5.0	8
<code>firebase_remote_config</code>	5.5.0 → 6.5.6	5
<code>firebase_analytics</code>	11.6.0 → 12.4.6	3
<code>cloud_firestore</code>	5.6.12 → 6.8.0	2
<code>firebase_storage</code>	12.4.10 → 13.4.6	2
<code>firebase_auth</code>	5.7.0 → 6.5.7	1
<code>firebase_performance</code>	0.10.1 → 0.11.4	—
<code>firebase_in_app_messaging</code>	0.8.1 → 0.9.2	—
<code>firebase_app_installations</code>	0.3.3 → 0.4.2	—

Maior bloco de risco do projeto

`firebase_core` 3→4 é migração coordenada: todos os pacotes Firebase têm que subir na mesma leva, senão o resolver quebra. Afeta crash reporting, push, analytics e remote config — coisas que **só aparecem em produção**.

Bloco B · Permissões e plataforma

RISCO ALTO

PACOTE	ATUAL → ALVO	ARQUIVOS
permission_handler	11.4.0 → 13.0.0 (2 majors)	4
flutter_local_notifications	18.0.1 → 22.2.0 (4 majors!)	2
local_auth	2.3.0 → 3.0.2	1
geolocator	13.0.4 → 14.0.2	2
connectivity_plus	6.1.5 → 7.3.1	2
workmanager	0.7.0 → 0.10.7	1
camera	0.11.2 → 0.12.0	3

`flutter_local_notifications` pulando 4 majors (18→22) e `permission_handler` 2 majors são os mais delicados. `workmanager` **é o que roda o sync de ponto offline do colaborador** — já identificado como crítico no plano de testes (cenário GERAL-04).

Bloco C · UI e widgets

RISCO MÉDIO

PACOTE	ATUAL → ALVO	ARQUIVOS
another_flushbar	1.12.32 → 2.2.4	18
f1_chart	0.70.2 → 1.2.0	5
image_cropper	9.1.0 → 12.2.1 (3 majors)	6
pdfrx	1.3.5 → 2.2.24	2
infinite_scroll_pagination	4.1.0 → 5.1.1	2
fluttertoast	8.2.14 → 9.0.0	2
dropdown_search	6.0.2 → 7.0.0	1
dotted_border	2.1.0 → 3.1.0	1
photo_view	0.14.0 → 0.15.0	1
flutter_widget_from_html_core	0.16.1 → 0.17.2	—

`another_flushbar` (18 arquivos) e `f1_chart` 0.70→1.0 (API de gráficos mudou bastante) são os que dão mais trabalho manual. Do lado positivo, são falhas **visíveis** — fáceis de detectar em teste.

Bloco D · Infra e utilitários

RISCO BAIXO-MÉDIO

PACOTE	ATUAL → ALVO	ARQUIVOS
<code>get_it</code>	8.3.0 → 9.2.1	4
<code>package_info_plus</code>	8.3.1 → 9.0.1	9
<code>device_info_plus</code>	11.5.0 → 12.4.0	3
<code>file_picker</code>	8.3.7 → 10.3.10	3
<code>flutter_contacts</code>	1.1.9 → 2.1.0	1
<code>datadog_flutter_plugin</code>	2.16.1 → 3.4.1	1
<code>datadog_tracking_http_client</code>	2.3.0 → 3.2.0	—
<code>google_mlkit_face_detection</code>	0.13.1 → 0.14.0	—

`get_it` 8→9 mexe na injeção de dependência — e o `application_container.dart` do `sindico` tem 3.500+ linhas. Vale atenção redobrada nesse.

Bloco E · Code generation

RISCO BAIXO

`build_runner`, `drift_dev`, `chopper_generator`, `json_serializable`, `source_gen`, `analyzer`, `lints`.

Depende do Blocker 2 — só é possível depois de limpar os overrides do `shared`. Após subir, todo código gerado precisa ser regenerado e commitado.

5. Plano de execução em 7 ondas

Estratégia

Uma **branch por onda**, cada uma testável e revertível de forma independente. **Nunca misturar blocos** — se algo quebra, precisamos saber exatamente qual bump causou.

#	ONDA	ESCOPO	ESFORÇO	RISCO
0	Preparação	Java 17 no sindaco, limpar overrides do <code>shared</code> , investigar <code>share_plus</code> / <code>open_file</code> , criar baseline	6-8h	Baixo
1	Patch + minor seguros	~150 pacotes sem breaking change via <code>pub upgrade</code>	3-4h	Muito baixo
2	Code generation	Bloco E + regenerar todo código gerado	4-6h	Baixo
3	Infra / utils	Bloco D (<code>get_it</code> , <code>package_info</code> , <code>device_info</code> , <code>file_picker</code> , <code>datadog</code>)	8-10h	Médio
4	UI	Bloco C (<code>flushbar</code> , <code>fl_chart</code> , <code>image_cropper</code> , <code>pdfx...</code>)	12-16h	Médio
5	Firebase	Bloco A (os 11 pacotes juntos)	16-20h	Alto
6	Permissões / plataforma	Bloco B (<code>permission_handler</code> , <code>notifications</code> , <code>workmanager...</code>)	12-16h	Alto
	TOTAL		60-80h	—

Detalhamento das ondas de maior atenção

Onda 0 — Preparação (fazer primeiro, sem exceção)

- Subir `sindico` para Java 17 e alinhar `compileSdk` com os outros dois apps.
- Remover `analyzer: 6.4.1` e `source_gen: 1.5.0` do `shared`. Rodar build para ver se o conflito original ainda existe.
- Investigar quem trava `share_plus` e `open_file` (`flutter pub deps`).
- Criar baseline:** build APK release dos 5 apps + `flutter test` + screenshots das telas críticas. É a régua de comparação de todas as ondas seguintes.
- Congelar `pubspec.lock` num commit dedicado antes de tudo.

Onda 1 — Ganho rápido

`flutter pub upgrade (sem --major-versions)`. Pega os 65 patches e os 85 minors não-breaking. Zero mudança de código esperada. Valida rápido se o pipeline continua verde e já derruba boa parte da dívida.

Onda 5 — Firebase (a mais crítica)

- Subir os 11 pacotes juntos, num **único commit**.
- Reconferir `google-services.json` / `GoogleService-Info.plist` e as configs dos **10 flavors**.
- Testar em device físico:** push notification chegando, crash forçado aparecendo no Crashlytics, remote config puxando valor novo.

- Firebase é justamente o que **falha silencioso**: o app compila, roda, e só semanas depois se descobre que crash não está sendo reportado.

Onda 6 — Permissões

- Testar cada permissão em **Android 13, 14 e 15** (as regras mudaram em cada uma) e **iOS 16+**.
- `flutter_local_notifications` 18→22: revisar canais de notificação e permissão `POST_NOTIFICATIONS` .
- `workmanager` : revalidar o **sync de ponto offline do colaborador** (cenário GERAL-04 do roteiro de testes).

6. Critérios de aceite por onda

Cada onda só é considerada concluída quando **todos** os itens abaixo passam:

CRITÉRIO	OK
<code>flutter analyze</code> = 0 erros nos 5 projetos	☐
<code>flutter test</code> sem regressão vs baseline da Onda 0	☐
<code>flutter build apk --release</code> OK nos 3 apps principais	☐
<code>flutter build ipa</code> OK (ou <code>pod install</code> + build no Xcode)	☐
Smoke test manual nos fluxos que o bloco toca	☐
White-labels (<code>morar-viver</code> , <code>sindico-viver</code>) compilando	☐
<code>pubspec.lock</code> commitado	☐
Branch da onda revisada em PR antes de merge	☐

7. Riscos principais

1. Firebase 4.x pode exigir Flutter mais novo

Se der conflito de SDK, entra na conta um upgrade de Flutter — que é outro projeto, com outro plano. Só descobrimos isso ao executar a Onda 5.

2. Oito pacotes têm `latest` > `resolvable`

`device_info_plus` , `file_picker` , `flutters.toast` , `package_info_plus` , `pdfx` , `flutter_contacts` , `share_plus` , `open_file` . Significa que o Flutter atual limita o teto. Vamos até o `resolvable` e o resto fica pendente de upgrade do Flutter.

3. Cinco apps de uma vez

Qualquer bump no `essentials` afeta todos simultaneamente. Regressão em um app pode passar batido se o time só testar os outros. O smoke test precisa cobrir os 5.

4. White-labels esquecidos

`morar-viver` e `sindico-viver` quase nunca são testados, mas herdam todas as libs. Incluir obrigatoriamente no critério de aceite.

8. Recomendação e próximos passos

Começar por Onda 0 + Onda 1 juntas

Em aproximadamente **10 horas** você tem: os blockers estruturais resolvidos, cerca de **150 pacotes atualizados**, e um baseline confiável para comparar as ondas seguintes. Isso derruba boa parte da dívida **sem risco de breaking change**, e revela cedo se o Flutter atual vai virar gargalo.

Depois de fechar as duas primeiras ondas, avaliar com o time se vale seguir para as ondas 2 a 6 de forma contínua ou parcelar ao longo de sprints — considerando que as ondas 5 e 6 exigem janela de teste em devices físicos e múltiplas versões de Android/iOS.

Sequência sugerida de aprovação

1. Aprovar este plano e a alocação de ~10h para Onda 0 + 1.
2. Executar, validar e fazer merge.
3. Reavaliar cronograma para as ondas 2-6 com dados reais de esforço das duas primeiras.

9. Anexo — comandos úteis

Diagnóstico

```
# Ver o que está desatualizado (por projeto)
flutter pub outdated

# Exportar em JSON para análise
flutter pub outdated --json > outdated.json

# Descobrir quem restringe a versão de um pacote
flutter pub deps | Select-String "share_plus"
```

Atualização

```
# Onda 1 – só patch e minor compatíveis (seguro)
flutter pub upgrade

# Major versions (usar só nas ondas 3 a 6, pacote por pacote)
flutter pub upgrade --major-versions <pacote>

# Após mexer no essentials, propagar nos apps
flutter pub get
```

Validação

```
# Analyzer
flutter analyze

# Testes
flutter test

# Builds
flutter build apk --flavor lelloMorar --release
flutter build apk --flavor lelloColaborador --release
flutter build bundle

# iOS (no macOS)
cd ios && pod install && cd ..
flutter build ipa
```

Code generation (Onda 2)

```
flutter pub run build_runner build --delete-conflicting-outputs
```

Documentos relacionados

- 01-testes-negocio/ — roteiros de teste para o time de negócio (Morar e Colaborador)
- 02-testes-tecnico/ — planos de teste técnicos da migração bloc 9

Os roteiros de teste existentes podem ser reaproveitados como base de validação das ondas 3 a 6.